DevOps with Pulumi

K19G

2026-11-09

Table of contents

DevOps with Pulumi **7**
What this book covers . . . 7
How the content is organized . . . 7
Formats . . . 8
Prerequisites . . . 8
Acknowledgments . . . 8

Intro **9**

What Pulumi is (and is not) **10**

What Pulumi is (and is not) **11**
Thesis . . . 11
Mental model . . . 11
What Pulumi is . . . 12
What Pulumi is not . . . 13
How the pieces show up day to day . . . 14
Common mistakes . . . 14
A tiny glossary you will keep meeting . . . 14
Why languages matter here . . . 15
Where to go next in foundations . . . 15
Summary . . . 15
Sources . . . 15

Foundations **16**

Install the CLI, create a project, and run your first preview **17**

Install the CLI, create a project, and run your first preview **18**
Learning goals . . . 18
Installing the Pulumi CLI on Linux . . . 18
Option 1: Official install script . . . 18
Option 2: Package managers . . . 19
Verifying installation . . . 19
Backend selection and authentication . . . 19
Pulumi Cloud (Default) . . . 20
DIY backends . . . 20
Creating a Python Pulumi project . . . 20
Generating a random provider project without cloud credentials . . . 21

Understanding the project structure . . . 21
The manifest: `Pulumi.yaml` . . . 21
Python dependencies: `requirements.txt` . . . 22
Infrastructure code: `__main__.py` . . . 22
Initializing stacks: `pulumi stack init` . . . 22
Running your first preview (`pulumi preview`) . . . 23
The deployment and destruction lifecycle . . . 23
Applying infrastructure changes (`pulumi up`) . . . 23
Tearing down infrastructure (`pulumi destroy`) . . . 24
Common mistakes and pitfalls . . . 24
1. Missing virtual environment activation . . . 24
2. Treating resource outputs as plain Python strings . . . 24
3. Modifying state files manually . . . 25
Summary . . . 25
Sources . . . 25

Projects, Stacks and State **26**

Projects, Stacks and State **27**
Learning goals . . . 27
Projects: the program and its `Pulumi.yaml` . . . 28
Stacks: many instances of one program . . . 28
Names . . . 29
Renaming has consequences . . . 29
Removing . . . 29
Outputs: what a stack publishes . . . 29
Stack references: wiring projects together . . . 30
State and backends . . . 31
Drift and refresh . . . 31
Raw state . . . 32
Diagnosis . . . 32
Operational care . . . 32
Practice . . . 33
Recall card . . . 33
Sources . . . 33

Configuration and Secrets **34**

Configuration and Secrets **35**
Learning goals . . . 35
Configuration: per-stack key/value pairs . . . 35
Keys and namespaces . . . 36
Reading from code . . . 36
Structured values . . . 36
Project-level defaults . . . 37
Provider configuration . . . 37
Secrets . . . 38
Setting . . . 38

Secrets are outputs that remember . . . 38
Where secret-ness ends . . . 39
Secrets providers . . . 39
Diagnosis . . . 40
Operational care . . . 40
Practice . . . 41
Recall card . . . 41
Sources . . . 41

Inputs, Outputs and the Dependency Graph **42**

Inputs, Outputs and the Dependency Graph **43**
Learning goals . . . 43
Inputs . . . 43
Outputs . . . 44
Getting at the value . . . 44
`apply`: transform one output . . . 45
`Output.all`: combine several . . . 45
Helpers: skip the lambda . . . 45
Lifting: reach into nested outputs . . . 46
Two rules that prevent most bugs . . . 46
Resource identity . . . 46
Diagnosis . . . 46
Practice . . . 47
Recall card . . . 47
Sources . . . 47

Unit Testing Pulumi Programs with Mocks **48**

Unit Testing Pulumi Programs with Mocks **49**
Why test with mocks? . . . 49
The Pulumi mock runtime . . . 49
Registering mocks before program execution . . . 50
Async testing with `IsolatedAsyncioTestCase` . . . 50
Python 3.14 event loop requirements . . . 51
Lab: testing an AWS infrastructure stack . . . 51
Project dependencies . . . 51
The infrastructure program (v1 with security flaw) . . . 51
The test suite . . . 53
Running the tests against v1 . . . 54
Fixing the infrastructure flaw . . . 55
How mock outputs resolve . . . 56
Pitfalls . . . 57
Summary . . . 57
Sources . . . 58

Component Resources **59**

Component Resources **60**
Thesis . . . 60
Why components? (vs functions returning resources) . . . 60
The type token . . . 61
The component . . . 61
Code Walk-Through . . . 62
`register_outputs` and what breaks without it . . . 63
Options that do and do not apply . . . 63
Visualizing the resource tree . . . 64
Testing component resources . . . 64
Complete Test Suite . . . 64
Two Critical Testing Pitfalls . . . 66
Running the Tests . . . 66
Evolving a component resource . . . 67
Using Aliases for Component Renames . . . 67
State Migrations for Child Restructuring . . . 67
Common mistakes . . . 68
Practice . . . 68
Key takeaways . . . 69
Sources . . . 69

Workflows **70**

Automation API: drive pulumi up from Python **71**

Automation API: drive pulumi up from Python **72**
Thesis . . . 72
CLI vs. Automation API . . . 72
Key Abstractions: Workspace, Program, and Stack . . . 73
Defining an Inline Program . . . 73
Code Walk-Through . . . 75
Executing Updates and Reading Outputs . . . 75
Programmatic Destroy and Stack Teardown . . . 75
Key Considerations for Teardown . . . 76
Working with Existing Disk-Based Projects . . . 76
Common Pitfalls . . . 77
Key takeaways . . . 78
Sources . . . 78

Pulumi ESC: environments, secrets, and dynamic credentials **79**

Pulumi ESC: environments, secrets, and dynamic credentials **80**
Thesis . . . 80
Mental model . . . 80
An environment document . . . 81
Opening an environment for a stack . . . 81
ESC vs stack secrets (chapter 03) . . . 82
Common mistakes . . . 82

Exercises 82
Summary 83
Sources 83

DevOps with Pulumi

> **Note**
>
> Replace this preface after you copy the template. Rename the book folder (e.g. `books/My-Topic`); the folder name becomes the Quarto book title when you run `scripts/update-index.sh`.

What this book covers

Summarize the audience, goals, and scope in a few bullets:

- Who this is for
- What you will be able to do
- What is deliberately out of scope

How the content is organized

Source lives under `content/`. Each first-level directory is a **part** in the sidebar:

Folder	Role
`content/00-intro/`	Orientation and overview
`content/01-foundations/`	Core material
`content/99-appendices/`	Optional extras (add when needed)
`content/_planning/`	Notes only — ignored by the index script

Chapter order is filename order. Prefer numeric prefixes:

```
content/01-foundations/01-getting-started.qmd
content/01-foundations/02-core-ideas.qmd
```

Set sidebar titles with YAML frontmatter:

```
---
title: "Getting Started"
---
```

Formats

This book is published as part of the monorepo library portal:

- **HTML** — primary reading experience (light/dark themes)
- **PDF / EPUB** — generated on full library builds

Prerequisites

List tools, background knowledge, or environment setup here.

Acknowledgments

Add credits here.

Intro

What Pulumi is (and is not)

What Pulumi is (and is not)

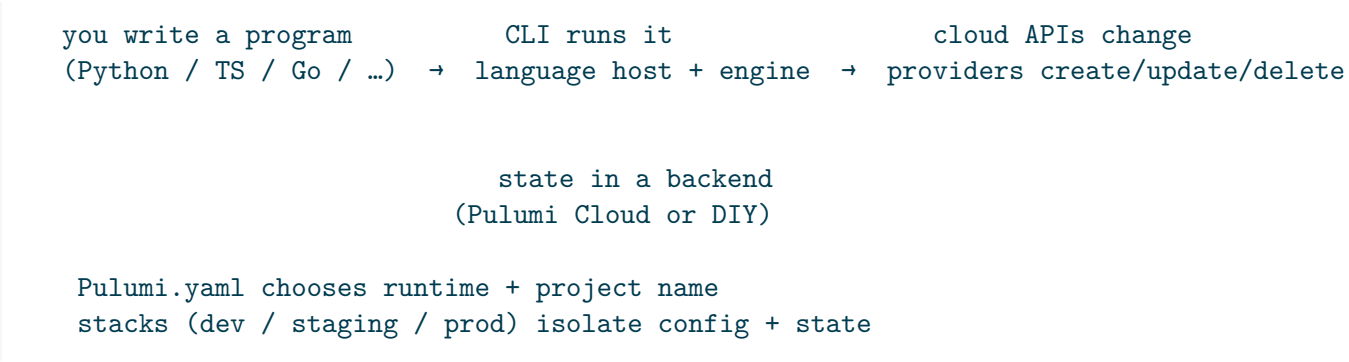
After this page you can explain what Pulumi is, name the four pieces every deployment uses (program, project, stack, backend/state), and say how those pieces differ from a shell script that calls cloud APIs or from a tool that only speaks a domain-specific language. Later foundations pages cover install, projects, stacks, config, inputs and outputs, tests, and components in depth. This page is the map, not the walkthrough.

Thesis

Pulumi is an infrastructure-as-code platform: you write a program in a general-purpose language (Python, TypeScript, Go, .NET, Java) or in YAML/HCL, declare the cloud resources you want, and the CLI drives a deployment engine that creates, updates, or deletes them until the live world matches the program. The engine records what it did in **state** stored in a **backend**. Each independently configured copy of the program is a **stack**.

That combination — ordinary languages, typed SDKs, a CLI, and durable state — is the product. Everything else in this book is a refinement of those four ideas.

Mental model



Piece	What it is	What breaks if you get it wrong
Program	Source that constructs resource objects	Wrong properties → wrong cloud objects
Project	Directory with `Pulumi.yaml` (runtime + name)	Wrong runtime → the language host never starts

Piece	What it is	What breaks if you get it wrong
Stack	One isolated instance of the program	Selecting `prod` while editing `dev` config
Backend / state	Record of what already exists	Lost or shared state → duplicate or orphan resources
Provider	Plugin that talks to one cloud or service	Missing credentials → every create fails

Pulumi tracks dependencies between resources from the values you pass (an instance that receives a security-group id depends on that group). The engine uses those edges to order creates and deletes and to parallelize where it can.

What Pulumi is

From the concepts documentation:

- **Multi-language.** TypeScript/JavaScript, Python, Go, .NET, Java, Pulumi YAML, and HCL are first-class. The same AWS security group and instance can be declared in any of them; the engine sees the same resource graph.
- **SDK + CLI + engine.** The language SDK constructs resources. The CLI (`pulumi up`, `pulumi preview`, …) orchestrates. The deployment engine diffs desired state against recorded state and calls providers.
- **Optional managed backend.** Pulumi Cloud stores state, adds access control, secrets, policy, and drift detection. The open-source CLI also works with self-managed backends (S3, Azure Blob, GCS, local files).
- **Open source core.** The CLI and SDKs are free and open source; Pulumi Cloud is the optional hosted service.

A minimal Python sketch (AWS) looks like this — the point is the shape, not a runnable lab on this box:

```
import pulumi
import pulumi_aws as aws

group = aws.ec2.SecurityGroup(
    "web-sg",
    description="Enable HTTP access",
    ingress=[
        {
            "protocol": "tcp",
            "from_port": 80,
            "to_port": 80,
            "cidr_blocks": ["0.0.0.0/0"],
        }
```

```
    ],
)

server = aws.ec2.Instance(
    "web-server",
    ami="ami-0319ef1a70c93d5c8",
    instance_type="t2.micro",
    vpc_security_group_ids=[group.id],
)

pulumi.export("public_ip", server.public_ip)
pulumi.export("public_dns", server.public_dns)
```

`group.id` is an **output**. Passing it into the instance wires a dependency: the group is created before the instance. Exports become stack outputs you can read from the CLI or from another stack.

> **Not run here**
>
> This authoring box has no `pulumi` CLI and no cloud credentials. The sketch follows the official concepts documentation; it was not previewed or deployed locally.

What Pulumi is not

Mistaken for	Reality
A shell script that calls `aws` / `az` / `gcloud`	Scripts are imperative and usually forget what they created. Pulumi is declarative over a state file: re-running converges.
"Just Terraform with Python syntax"	Pulumi can use HCL and can bridge providers, but the programming model is the language SDK (classes, functions, real package managers), not a DSL with a foreign expression language.
A replacement for your cloud console	You still need accounts, IAM, and provider credentials. Pulumi drives APIs; it does not replace billing or identity.
A guarantee that preview equals apply	Preview is best-effort. Some providers resolve values only at apply time; always read the plan, and treat apply as the source of truth.

Mistaken for	Reality
Something that invents credentials	Providers need the same env vars, profiles, or OIDC setup your CLI tools need. Missing creds fail at preview/up, not at `pulumi new`.

How the pieces show up day to day

1. **Author** a program in a project directory.
2. **Select** a stack (`dev`, `staging`, `production`, or a feature stack).
3. **Preview** the diff against that stack's state.
4. **Up** to apply, or **refresh** if the cloud drifted outside Pulumi.
5. **Destroy** when the stack should go away (and only then).

Config and secrets are per-stack. Inputs and outputs carry values between resources and between stacks. Component resources group children under one parent type. Unit tests mock the engine so you can assert on the resource graph without touching a cloud. Each of those topics has its own foundations page; you do not need them to understand the overview.

Common mistakes

- **One state file, many humans, no locking story.** A DIY backend without locking invites two applies at once. Pulumi Cloud handles this; DIY backends need you to configure it.
- **Treating stacks as branches of the code.** Stacks share the program; they differ in config and state. Feature work usually means a new stack or a branch of the *code*, not a fork of state.
- **Editing state by hand.** Export/import and `pulumi state` exist for disasters. Day-to-day fixes belong in the program plus `up`, or in `refresh` when the cloud moved first.
- **Assuming language features equal cloud features.** A Python `for` loop that creates 50 buckets still creates 50 remote objects and 50 state entries. Abstraction in code does not reduce cloud cost by itself.

A tiny glossary you will keep meeting

- **Resource.** One cloud object (or logical object) with a type token, a logical name in the program, and properties.
- **Provider.** The plugin that implements create/read/update/delete for a family of types (`aws`, `azure-native`, `random`, …).
- **Output.** A value that may not be known until apply time (an allocated id, a DNS name). You pass outputs into other resources; you cannot treat them like plain strings without `apply` (covered in inputs/outputs).
- **Preview / up / destroy / refresh.** Plan; apply; delete stack resources; reconcile state with the live cloud.

- **Stack reference.** One stack reading another stack's exports — how you split "network" from "app" without copying ids by hand.

Why languages matter here

Because the program is real Python (or Go, or TypeScript), you get ordinary packaging, ordinary tests, and ordinary editor support. That is the point of Pulumi relative to a pure DSL: reuse modules, share libraries inside a company, and unit-test the resource graph with mocks. The cost is discipline — a clever loop can still create hundreds of expensive objects — so reviews look at both the code and the preview.

Where to go next in foundations

Install the CLI and run a first preview. Then learn projects and stacks (where state lives), configuration and secrets (per-stack values), inputs and outputs (dependencies), unit testing with mocks, and component resources (typed groups of children). Skip ahead only if you already have a working CLI and a backend login.

Summary

- Pulumi declares cloud resources in ordinary languages (or YAML/HCL), then converges live infrastructure to that declaration through a CLI, an engine, and providers.
- **Project** (program + `Pulumi.yaml`), **stack** (isolated instance), and **backend/state** (what already exists) are the three nouns you must keep straight.
- It is not a shell script, not "credentials included," and not a promise that preview is identical to apply.
- The rest of the foundations material builds on this map: install and first preview, then projects/stacks, config, inputs/outputs, tests, and components.

Sources

- Pulumi concepts
- Projects
- Stacks
- State and backends

Foundations

Install the CLI, create a project, and run your first preview

Install the CLI, create a project, and run your first preview

After reading this chapter, you will be able to install the Pulumi CLI on Linux, select and authenticate with a state backend, set up a minimal Python infrastructure project using standard virtual environments, inspect its generated entrypoints, run a safe preview, and manage resource lifecycles through execution and destruction.

> **Not run here**
>
> The authoring environment does not have the `pulumi` CLI installed and system-level installation was disabled. The commands, output structures, and source listings in this chapter follow official Pulumi CLI version 3.x specifications and documentation guidelines. None of the shell commands below were executed on the local host.

Learning goals

- Install the Pulumi CLI on Linux using official repository scripts or package managers
- Differentiate between Pulumi Cloud and self-managed (DIY) state backends
- Bootstrap a Python Pulumi project and explain `Pulumi.yaml`, `requirements.txt`, and `__main__.py`
- Initialize stack environments and run `pulumi preview` to inspect state changes before applying them
- Execute the full infrastructure lifecycle using `pulumi up` and `pulumi destroy` safely

Installing the Pulumi CLI on Linux

Pulumi delivers infrastructure as code by running standard programming languages against a CLI binary (`pulumi`). The CLI translates your code into resource registrations, sends them to provider plugins (such as AWS, Azure, Google Cloud, or utility providers like Random), and records state transitions.

On Linux systems, you can install the Pulumi binary via the official installation script, through package managers, or by manually downloading release archives.

Option 1: Official install script

The primary method to install the latest CLI binary is running the official shell script:

```
curl -fsSL https://get.pulumi.com | sh
```

The script installs the `pulumi` executable into `$HOME/.pulumi/bin`. To ensure the executable is accessible across your terminal sessions, append that directory to your system `PATH` in your shell configuration (`~/.bashrc` or `~/.zshrc`):

```
export PATH=$PATH:$HOME/.pulumi/bin
```

Option 2: Package managers

If your distribution manages packages via Homebrew or standard system repositories, install Pulumi directly:

```
# Using Homebrew on Linux
brew install pulumi
```

Verifying installation

After installing, confirm the version and verify that the executable resolves properly:

```
pulumi version
```

This command prints the active version string (for example, `3.274.0`).

Backend selection and authentication

Before managing infrastructure, Pulumi requires a **backend** to store stack state. State tracks the mapping between declared code resources and live backend infrastructure instances.

Pulumi supports two primary backend styles: **Pulumi Cloud** (managed service) and **Self-managed DIY backends** (local filesystem or cloud object storage).

Backend Type	Login Command	State Storage Location	Encryption & Secrets	Best Used For
Pulumi Cloud	`pulumi login`	Managed cloud service	Automatic managed key encryption	Production teams, shared state, audit logs
Local Filesystem	`pulumi login file://~`	`~/.pulumi/workspaces`	Local passphrase or KMS	Local testing, single-user experiments
S3 / Blob Storage	`pulumi login s3://my-bucket`	AWS S3 / GCS / Azure Blob	Customer-managed KMS or passphrase	Existing cloud storage policies without Pulumi Cloud

Pulumi Cloud (Default)

Running `pulumi login` without arguments authenticates your terminal against the Pulumi Cloud service.

> **Note**
>
> **Not run here.** Executing `pulumi login` opens a browser window or prompts for an access token to authenticate with `app.pulumi.com`.

```
pulumi login
```

Upon entering a valid personal or organization access token, Pulumi saves credential tokens into `~/.pulumi/credentials.json`.

DIY backends

If your compliance model requires self-managed state or offline operations, specify a URI scheme pointing to local disk or remote object stores:

```
# Local filesystem backend
pulumi login file://~

# AWS S3 backend
pulumi login s3://my-company-pulumi-state-bucket
```

To switch back to Pulumi Cloud later or query your current backend identity, use `pulumi whoami`:

```
pulumi whoami -v
```

Creating a Python Pulumi project

A Pulumi **project** is a directory containing a `Pulumi.yaml` manifest and executable source files. Project creation is driven by `pulumi new`.

Create an empty working directory and launch the initialization wizard:

```
mkdir my-first-pulumi && cd my-first-pulumi
pulumi new python --yes
```

The `--yes` flag accepts default configuration values for project name, description, and initial stack name (`dev`).

Generating a random provider project without cloud credentials

To learn Pulumi without configuring AWS, Azure, or GCP credentials, create a project centered on utility resources like the `random` provider.

Initialize a standard Python template using non-interactive flags:

```
pulumi new python \
    --name demo-random \
    --description "A minimal Python Pulumi project using the random provider" \
    --stack dev \
    --yes
```

This command sets up virtual environments, installs core SDK dependencies, creates configuration files, and creates an initial active stack named `dev`.

Understanding the project structure

A standard Python Pulumi directory contains the following core files:

```
my-first-pulumi/
   Pulumi.yaml
   Pulumi.dev.yaml
   requirements.txt
   venv/
   __main__.py
```

The manifest: `Pulumi.yaml`

`Pulumi.yaml` defines project-level metadata, runtime execution rules, and program entrypoints.

```
name: demo-random
runtime:
  name: python
  options:
    virtualenv: venv
description: A minimal Python Pulumi project using the random provider
```

Key fields include: - `name`: Unique identifier for the program. Used to namespace project outputs and configurations. - `runtime`: Defines language bindings (`python`, `nodejs`, `go`, `dotnet`, `java`, `yaml`). The optional `virtualenv` option tells Pulumi which Python virtual environment directory to activate automatically during CLI operations. - `description`: Freeform text summary of project purpose.

Python dependencies: `requirements.txt`

The default `requirements.txt` includes the primary Pulumi Python SDK:

```
pulumi>=3.0.0,<4.0.0
pulumi-random>=4.0.0,<5.0.0
```

Install these dependencies into your project virtual environment using standard Python package management tools:

```
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
```

Infrastructure code: `__main__.py`

The entrypoint file `__main__.py` contains Python code that builds infrastructure declaratively. Replace the generated stub in `__main__.py` with code that generates a random password string and exports its result.

```
import pulumi
import pulumi_random as random

# Declare a random password resource
password = random.RandomPassword(
    "api-token",
    length=16,
    special=True,
    override_special="_@#%"
)

# Export values to make them available to stack outputs
pulumi.export("password_result", password.result)
```

In this code: 1. `random.RandomPassword` declares a high-entropy string resource managed by Pulumi. 2. `"api-token"` is the **logical name** assigned inside Pulumi state. 3. `pulumi.export` publishes calculated attributes to stack outputs when deployments complete.

Initializing stacks: `pulumi stack init`

A **stack** is an isolated deployment target of your project (e.g., `dev`, `staging`, `prod`). While `pulumi new` creates an initial stack automatically, you can create additional stacks using `pulumi stack init`:

```
pulumi stack init staging
```

To list all stacks registered under the current project:

```
pulumi stack ls
```

To select an active stack for subsequent CLI commands:

```
pulumi stack select dev
```

Running your first preview (`pulumi preview`)

`pulumi preview` performs a dry run of your program. It executes your Python script, computes differences between declared code resources and existing state snapshot records, and lists proposed additions, updates, or deletions without modifying real-world resources.

Run a preview against the `dev` stack:

```
pulumi preview
```

> **Note**
>
> **Not run here.** Running `pulumi preview` output presents a execution plan diff detailing actions Pulumi will execute upon deployment.

During a preview: 1. Pulumi launches the Python runtime and executes `__main__.py`. 2. Provider plugins register requested resources against the engine core. 3. The engine computes structural changes relative to existing state. 4. Pulumi displays a summary table listing planned operations (`+ create`, `~ update`, `- delete`).

The deployment and destruction lifecycle

Deploying infrastructure involves two complementary commands: `pulumi up` to create/converge resources, and `pulumi destroy` to tear down resources managed by the stack.

Applying infrastructure changes (`pulumi up`)

To apply proposed resource changes to your target environment, run `pulumi up`.

```
pulumi up --yes
```

> **Note**
>
> **Not run here.** Executing `pulumi up` provisions live infrastructure according to the calculated plan and saves updated resource attributes into the active backend state file.

When `pulumi up` runs: 1. Pulumi performs a preview diff. 2. The engine calls provider APIs to construct declared resources (`random:index:RandomPassword`). 3. Upon success, resulting attributes (such as `result`) are saved to state. 4. Top-level `pulumi.export` entries display in stdout output.

To inspect exported stack values after deployment, use `pulumi stack output`:

```
pulumi stack output password_result --show-secrets
```

Tearing down infrastructure (`pulumi destroy`)

When infrastructure is no longer required, remove all resources managed by the active stack using `pulumi destroy`.

> **Note**
>
> **Not run here.** `pulumi destroy` reads the state snapshot, calculates reverse-order deletion dependencies, and issues removal operations to cloud APIs.

```
pulumi destroy --yes
```

Once all resources are removed from the live environment, delete the empty stack definition and its configuration history:

```
pulumi stack rm dev --yes
```

Common mistakes and pitfalls

Avoid these common issues when starting out with Pulumi in Python environments:

1. Missing virtual environment activation

If Pulumi fails to import `pulumi` or provider packages during `pulumi preview`, check `Pulumi.yaml`. Ensure `runtime.options.virtualenv` points to your active virtual environment folder (`venv`), or activate your virtual environment in your shell prior to executing CLI commands.

2. Treating resource outputs as plain Python strings

Pulumi resource properties are asynchronous `Output[T]` wrappers, not standard Python data types (`str`, `int`).

```
# INCORRECT: Treating Output[str] as a standard string
# This concatenates the string representation of the Output object, not its value.
```

```
db_url = "https://" + password.result

# CORRECT: Use apply() or Output.concat() to construct derived values
db_url = password.result.apply(lambda pwd: f"https://admin:{pwd}@example.com")
```

3. Modifying state files manually

Never manually edit `.json` state files created by DIY backends or downloaded via `pulumi stack export`. Direct state editing bypasses transaction validation and risks corrupting infrastructure mappings. Use `pulumi state` subcommands (`pulumi state delete`, `pulumi state unprotect`) if resource state corrections become necessary.

Summary

- The Pulumi CLI translates declarative code into concrete API actions using provider plugins.
- `pulumi login` switches backend state targets between Pulumi Cloud and DIY options like `file://~` or `s3://`.
- `pulumi new python` generates standard project skeletons consisting of `Pulumi.yaml`, `requirements.txt`, and `__main__.py`.
- `pulumi preview` performs safe, dry-run diff computations comparing current code against saved stack state.
- `pulumi up` executes infrastructure creation or updates; `pulumi destroy` tears down managed resources cleanly.

Sources

- Pulumi Download and Installation Guide
- Pulumi Projects Concept Documentation
- Pulumi CLI Command Reference: `pulumi new`
- Pulumi CLI Command Reference: `pulumi preview`

Projects, Stacks and State

Projects, Stacks and State

Every Pulumi command you run answers three questions before it touches a cloud API: *which program* (the project), *which copy of it* (the stack), and *where is the record of what already exists* (the backend and its state). Get those three straight and the rest of Pulumi is "write a program that declares resources". Get them wrong and you deploy staging's code into production's state.

> **What was and was not run**
>
> The authoring box had no `pulumi` CLI and the run rules forbid installs, so none of the commands below were executed. Command lines, file names and behaviour follow the current Pulumi documentation. The CLI listings are reproduced from that documentation, not captured locally, and are marked as such. Examples use Python. Every other Pulumi language has an equivalent call, listed in the docs.

Learning goals

- Read a `Pulumi.yaml` and say which runtime and entrypoint it selects
- Create, select, rename and remove stacks, and explain fully qualified stack names
- Export stack outputs and consume them from another project with a `StackReference`
- Choose between Pulumi Cloud and a DIY backend, and know what `pulumi refresh` does and does not do

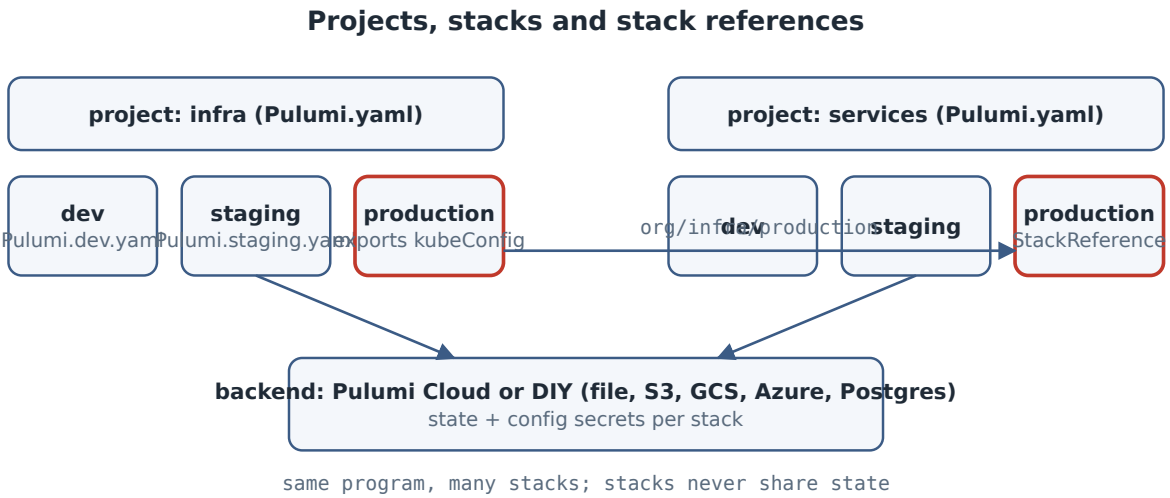


Figure 1: Two projects, three stacks each, linked by stack references; every stack has its own state in the backend.

Projects: the program and its `Pulumi.yaml`

A **project** is any directory containing a `Pulumi.yaml`. At run time Pulumi walks up from the current directory to the nearest one, and that file decides the project. `pulumi new` creates it.

```yaml
name: webserver
runtime: python
description: A minimal Pulumi program.
```

- `name` is the project name. It also namespaces configuration keys (next chapter).
- `runtime` is one of `nodejs`, `python`, `go`, `dotnet`, `java`, `yaml` or `hcl`.
- Each runtime finds its entrypoint by its own convention: `__main__.py` (or `setup.py`) for Python, `package.json` for Node, the build tool for Go, .NET and Java, `.tf` files for HCL.

The file name must start with a capital `P`. Either `.yaml` or `.yml` works.

For compiled languages you can skip the build at deploy time by pointing at a prebuilt binary:

```yaml
name: my-project
runtime:
  name: go
  options:
    binary: mybinary
```

Paths. File paths inside a program are relative to the *working directory*. If the program must find files next to `Pulumi.yaml` regardless of where it was launched, use `pulumi.get_root_directory()` and convert the result to a path relative to the working directory, so plans don't differ between machines.

Stacks: many instances of one program

A **stack** is an isolated, independently configured instance of the project's program, typically `dev`, `staging`, `production`, or one per feature branch. Each stack has its own configuration and its own state. The program is shared.

```
pulumi stack init staging     # create and select
pulumi stack ls               # list stacks of the nearest project
pulumi stack select dev       # change the active stack
pulumi stack                  # show the active stack's resources and outputs
```

The commands `config`, `preview`, `up` and `destroy` act on the **active** stack. Pass `--stack` (or `-s`) in scripts and pipelines rather than relying on whatever was selected last.

Names

Stack names may contain letters, digits, hyphens, underscores and periods, and must be unique within a project. A stack can be written three ways:

Form	Meaning
`dev`	Current user or default organization, nearest project
`acme/dev`	Organization `acme`, nearest project
`acme/webserver/dev`	Fully qualified: organization, project, stack

On a DIY backend the organization part is always the literal `organization`. Backends initialised by a CLI older than v3.61.0 only understand the short form until upgraded with `pulumi state upgrade`.

Renaming has consequences

`pulumi stack rename production` changes the value of `pulumi.get_stack()`. If your program uses the stack name in resource names, the next `pulumi up` proposes **replacements**. Read the preview. Sometimes creating a new stack is the safer operation.

A rename across projects (`acme/old-project/prod` to `acme/new-project/prod`) needs two manual follow-ups: change `name` in `Pulumi.yaml`, and rename project-namespaced keys in `Pulumi.<stack>.yaml` (for example `old-project:database` to `new-project:database`). Skip the second step and the program silently sees those values as unset.

Removing

`pulumi destroy` deletes the stack's resources. `pulumi stack rm` then deletes the empty stack, its history and its `Pulumi.<stack>.yaml`. `pulumi stack rm --force` removes a stack that still has resources, orphaning them. Do not.

Outputs: what a stack publishes

A stack exports values from the top level of the program:

```
import pulumi

pulumi.export("x", "hello")
pulumi.export("o", {"num": 42})
```

From the documentation (not run here), reading them back looks like:

```
$ pulumi stack output x
hello
```

```
$ pulumi stack output --json
{
  "x": "hello",
  "o": {
      "num": 42
  }
}
```

Outputs are effectively JSON-serialised and resolved after the update finishes. Export IDs and names, not whole resource objects, which serialise to large blobs. Secret outputs display as `[secret]` unless you pass `--show-secrets`.

Stack references: wiring projects together

Large systems split into projects by lifecycle and ownership, for example `infra` (network, cluster) and `services` (apps deployed onto it). The pairing is by environment: `acme/infra/production` feeds `acme/services/production`. A `StackReference` reads another stack's outputs:

```
import pulumi
from pulumi import StackReference

env = pulumi.get_stack()
infra = StackReference(f"acme/infra/{env}")

kubeconfig = infra.require_output("kubeConfig")
```

Use the fully qualified name. Pulumi records the dependency between stacks. Choose the reader carefully:

Method	Behaviour	Use
`require_output`	Fails the deployment if the output is missing	Default choice; turns a typo into an error
`get_output`	Returns an output wrapping `None` if missing	Only when absence is expected and handled
`get_output_details`	Awaitable; returns `.value` or `.secret_value` directly	When program logic needs the raw value. Python needs an async entrypoint and SDK 3.254.0 or later

The referenced stack's outputs are its public interface. Renaming an output is a breaking change for every consumer, just like renaming a function.

State and backends

Each stack's **state** is Pulumi's record of the resources it manages. `preview` and `up` diff the program against that record, not against the live cloud. The state lives in a **backend**:

- **Pulumi Cloud** (the default, also self-hostable): `pulumi login` with no argument. Updates go through a transactional API with full history.
- **DIY**: object storage you run (S3 or compatible such as MinIO or Ceph, Azure Blob, GCS), a PostgreSQL database, or the local filesystem. `pulumi login <backend-url>`, for example `pulumi login file://~` for local files.

Avoid typing the URL every time by setting `PULUMI_BACKEND_URL` or a `backend.url` in `Pulumi.yaml`. `pulumi whoami -v` shows the user and the backend in use. Credentials for backends are stored in `~/.pulumi/credentials.json`. Your *cloud* credentials are never stored in state. They stay where the CLI runs.

With a DIY backend you take on access control, encryption at rest, backups and auditing yourself. Checkpoint history lives under `.pulumi/history/`, but blob storage is not transactional, so some partial failures cannot be recovered automatically the way Pulumi Cloud recovers them.

Moving a stack between backends is `pulumi stack migrate` (CLI v3.254.0 or later). Log in to the target and name the source:

```
pulumi login                            # target: Pulumi Cloud
pulumi stack migrate file://~ my-app-production
```

It copies the latest checkpoint and configuration, re-encrypts secrets with the target's secrets provider, and leaves the source untouched.

Drift and refresh

Pulumi does not query every resource on every run. That is a deliberate choice for speed, predictability and explicit control. A console edit is invisible until you run:

```
pulumi refresh              # update state from the cloud; changes no infrastructure
pulumi preview --refresh    # refresh, then show what up would do
pulumi up --refresh         # refresh, then converge to the program
```

After a refresh, the next `up` reverts out-of-band changes unless you also change the program to keep them.

Raw state

`pulumi stack export --file stack.json` and `pulumi stack import --file stack.json` give you the JSON checkpoint for inspection or emergency repair. An incorrect import can orphan resources. Export first, keep the copy, and prefer the `pulumi state` subcommands for targeted edits.

Diagnosis

Symptom	Likely cause	Check
`up` acts on the wrong environment	Active stack was not the one you assumed	`pulumi stack ls` (the `*` marks the active one); pass `--stack`
CLI says no project (`Pulumi.yaml`) was found	Running outside the project tree	`ls Pulumi.yaml` in the directory or a parent
Preview proposes replacing everything after a rename	Stack name used in resource names	Revert the rename, or accept replacement deliberately
Config values silently missing after rename	Project-namespaced keys still carry the old name	Inspect `Pulumi.<stack>.yaml`
StackReference fails	Wrong qualified name, missing permission, or output not exported	`pulumi stack output -s org/project/stack`
Stacks "disappeared"	Logged in to a different backend	`pulumi whoami -v`
Plan ignores a console change	No refresh	`pulumi preview --refresh`

Operational care

- One project per lifecycle and owner. One stack per environment. Do not encode environments as `if` statements on the stack name for anything bigger than sizes and counts.
- Commit `Pulumi.yaml` and the `Pulumi.<stack>.yaml` files of long-lived stacks (secrets in them are encrypted). Ephemeral stacks usually do not commit theirs.
- In CI, always pass `--stack` with the fully qualified name, and run `preview` in pull requests and `up` on merge.
- Tag stacks (`pulumi stack tag set environment production`) so Pulumi Cloud can group them. Avoid the reserved prefixes `pulumi:`, `gitHub:` and `vcs:`.
- On DIY backends: version the bucket, restrict who can write, and back it up. That bucket *is* your inventory.

Practice

1. Sketch the project and stack layout for a system with a shared network, a database, and three services, across dev and production. How many stacks, and which references?
2. Write the `StackReference` line a `services` program needs to read `vpcId` from the matching `infra` stack, and justify `require_output` over `get_output`.
3. List what you must do yourself if you choose an S3 DIY backend instead of Pulumi Cloud.

Recall card

- Project = directory with `Pulumi.yaml` (name, runtime). Stack = one configured instance with its own state.
- `org/project/stack` is the full name. DIY backends use the literal `organization`.
- `pulumi.export` publishes; `StackReference.require_output` consumes.
- Backend = Pulumi Cloud or DIY. `refresh` syncs state from reality, explicitly. `stack migrate` moves between backends.

Sources

- Pulumi, Projects
- Pulumi, Stacks
- Pulumi, State and backends

Configuration and Secrets

Configuration and Secrets

A stack differs from its siblings in exactly two places: its state and its **configuration**. Configuration is how one program becomes a `t3.micro` in dev and a `m7i.large` in production without a single `if`. Some of that configuration is a password, and Pulumi's **secrets** system makes sure the password is encrypted in every place it travels to, not just the file you typed it into.

> **What was and was not run**
>
> `pulumi` was not available on the authoring box and installs were not allowed, so no commands here were executed. Commands, file layouts and listings follow the current Pulumi documentation. Listings such as `[secret]` masking are reproduced from the docs and marked as such. Examples use Python.

Learning goals

- Set, read and structure stack configuration, and explain key namespaces
- Choose between `get`, `require` and the `*_secret` variants, and between stack-level and project-level config
- Explain how secret-ness propagates through outputs, and where it does not (resource IDs, `apply` callbacks)
- Pick and change a secrets provider, including for DIY backends

Configuration: per-stack key/value pairs

Values live in `Pulumi.<stack>.yaml` next to `Pulumi.yaml`, and you **commit** that file. The values drive the program, so they belong in version control alongside it.

```
pulumi config set aws:region us-west-2      # provider namespace
pulumi config set instanceCount 3           # project namespace (implicit)
pulumi config get instanceCount
pulumi config                               # list; --json for machines
cat id_ed25519.pub | pulumi config set publicKey   # stdin: no shell escaping
```

`config set` overwrites an existing value **without warning**. Review the diff of `Pulumi.<stack>.yaml` before committing.

Keys and namespaces

Keys are `[namespace:]name`. With no colon, the namespace is the project name. That is how `aws:region` and your own `region` coexist. In the stack file the result looks like:

```yaml
config:
  aws:region: us-west-2
  webserver:instanceCount: "3"
```

In code, `pulumi.Config()` reads the project namespace and `pulumi.Config("aws")` reads another one. Reusable components should read their *own* namespace (`pulumi.Config("mylib")`) so they never collide with the consuming project.

Reading from code

```python
import pulumi

config = pulumi.Config()
name = config.require("name")              # missing -> deployment stops with an explanatory
count = config.get_int("instanceCount") or 1   # missing -> None, so default it
db_password = config.require_secret("dbPassword")  # Output, marked secret

aws_region = pulumi.Config("aws").require("region")
```

Use `require` for anything without a sensible default. A typo in a key should fail the preview, not quietly create one instance in production. Configuration is **read-only** at run time. Writing config from code is the job of the Automation API.

Environment variables also reach the program (`os.environ`), but configuration is per-stack, versioned and visible to collaborators, so prefer it.

Structured values

`--path` treats the key as a path into an object:

```
pulumi config set --path 'data.active' true
pulumi config set --path 'data.nums[0]' 1
pulumi config set --path 'data.nums[1]' 2
```

```yaml
config:
  proj:data:
    active: true      # booleans stay booleans
    nums: [1, 2]      # integer-like values become integers
```

```
data = pulumi.Config().require_object("data")
active = data["active"]          # a plain dict, not a Config
```

`require_object` returns a plain dictionary. Index into it. You cannot call `require` on a nested value.

Project-level defaults

Values every stack shares go in `Pulumi.yaml`, and any stack can override them:

```
# Pulumi.yaml
config:
  aws:region: us-east-1
  name: BroomeLLC
  data:
    value:            # structured project-level values need this wrapper
      active: true
```

```
# Pulumi.dev.yaml: overrides for one stack
config:
  aws:region: us-east-2
  myproject:name: MopLLC
```

Watch the two syntaxes. The stack file uses `project:key` with structures nested directly. The project file uses bare `key` and wraps structures in `value:`. Project-level config is edited by hand (`config set` doesn't write it) and holds **plaintext only**.

The project file can also **type-check** stack values:

```
config:
  name:
    type: string
    description: Base name for resources.
    default: BroomeLLC
  subnets:
    type: array
    items:
      type: string
```

A stack that sets `name` to a number then fails at `pulumi preview`. Type specifications don't yet cover structured config.

Provider configuration

A provider takes settings from three places: `pulumi config set aws:region ...`, its environment variables, or constructor arguments. Only the **default** provider reads stack config. An explicit

`aws.Provider(...)` you create reads nothing from config, so pass its arguments explicitly. To forbid implicit default providers altogether:

```
config:
  pulumi:disable-default-providers: ["aws", "kubernetes"]   # or ["*"]
```

`pulumi:tags` in config applies stack tags on every `up` or `refresh`. Removing a key from config does *not* delete the tag from the stack.

Secrets

Setting

```
pulumi config set --secret dbPassword 'S3cr37'
```

The stack file now holds ciphertext:

```
config:
  myproject:dbPassword:
    secure: AAABAIIlW0ewSuZ1FJxw/+Rpw6BNqTUvGJ30O8WkpL2hB4aPyS7UU68=
```

Quote values containing `$`, `!` or `#`, or pipe them from a file, so the shell doesn't change them first. Secrets work inside structured config too: `pulumi config set --path --secret 'endpoints[0].token' ...`.

The inverse: the CLI tries to detect values that *look* like tokens and stops you. When it guesses wrong, use `--plaintext`.

As documented (not run here), listings and accidental prints are masked:

```
$ pulumi config
KEY                         VALUE
aws:region                  us-west-1
dbPassword                  [secret]
```

Secrets are outputs that remember

A secret is an `Output` with a flag. Pulumi encrypts it wherever it is persisted, and **propagates** the flag. Anything computed from a secret with `apply` or `Output.all` is also secret. Resource properties built from it are encrypted in state.

```
import pulumi
import pulumi_aws as aws

cfg = pulumi.Config()
```

```python
param = aws.ssm.Parameter(
    "a-secret-param",
    type="SecureString",
    value=cfg.require_secret("my-secret-value"),   # encrypted in state too
)
```

Ways to make a value secret:

Source	How
Config	`get_secret` / `require_secret`
An existing value	`pulumi.Output.secret(value)`
A resource's outputs	`ResourceOptions(additional_secret_outputs=[`
Derived values	Automatic via `apply` / `Output.all` on a secret

Reading a `--secret` key with plain `require` hands you the plaintext and **drops** the marking downstream. Always use the secret getters.

Where secret-ness ends

- **Inside `apply`.** The callback receives plaintext. Pulumi marks the *return value* secret, but cannot stop the callback from printing or writing it to a file.
- **Resource IDs.** A resource's `id` is always stored in plaintext, and `additional_secret_outputs=["id"]` does nothing. `random.RandomString` exposes its `result` as its ID too, so its value lands unencrypted in state. Use `random.RandomPassword` and mark `result` secret.
- **Outside the stack.** Secret-ness covers one stack's state. Sharing the same credential across stacks or teams is a job for Pulumi ESC environments, which a stack imports with an `environment:` list in its stack file. Explicit stack config wins over imported values.

Secrets providers

By default with Pulumi Cloud, each stack gets its own Cloud-managed key and each value a nonce. Choose another provider when you use a DIY backend or your organisation mandates its own KMS:

```
pulumi stack init prod --secrets-provider="awskms://alias/pulumi-prod?region=us-east-1"
pulumi stack init prod --secrets-provider="gcpkms://projects/P/locations/L/keyRings/R/crypto
pulumi stack init prod --secrets-provider="azurekeyvault://myvault.vault.azure.net/keys/pulu
pulumi stack init prod --secrets-provider="hashivault://pulumi-prod"
pulumi stack init prod --secrets-provider=passphrase
```

- `awskms` accepts an ID, alias or ARN, an optional `profile=`, and (CLI v3.41.1+) `context_<k>=<v>` encryption context you can condition IAM policies on.
- `hashivault` reads `VAULT_SERVER_URL` and `VAULT_SERVER_TOKEN`.

- `passphrase` derives the key from a passphrase you supply (typically through `PULUMI_CONFIG_PASSPHRASE`). Lose it and the stack's secrets are unrecoverable.

The provider's public bits (`encryptionsalt` for passphrase, `encryptedkey` for KMS-style providers) are written into `Pulumi.<stack>.yaml` and are safe to commit. To switch an existing stack:

```
pulumi stack change-secrets-provider "awskms://alias/pulumi-prod?region=us-east-1"
pulumi preview      # expect: no changes
```

This re-encrypts config and state. If you'd rather not commit stack files at all, `pulumi config refresh` rebuilds them from the last deployment.

Diagnosis

Symptom	Likely cause	Fix
"Missing required configuration variable"	Key not set for *this* stack, or wrong namespace	`pulumi config -s <stack>`; check the `project:` prefix
Value set, program says unset	Set under a different stack, or project renamed	Confirm the active stack; rename keys in the stack file
Password visible in `stack export`	Read with `require` instead of `require_secret`, or carried in an `id`	Use the secret getters; avoid ID-bearing resources
CLI refuses a harmless value as "looks like a secret"	Secret detection heuristic	`--plaintext`
"passphrase must be set" or decrypt fails in CI	Passphrase or KMS access missing in the runner	Provide `PULUMI_CONFIG_PASSPHRASE` or KMS credentials
Preview fails with a type error	Project-level type spec	Fix the stack value's type

Operational care

- Commit `Pulumi.yaml` and `Pulumi.<stack>.yaml`. Ciphertext is meant to be committed. Never commit plaintext you meant to mark secret, because git history keeps it.
- Treat `pulumi config set` in pipelines as a code change. Prefer reviewing the stack file in a PR to mutating config during deploys.
- Use one KMS key or ESC environment per trust boundary (prod versus non-prod), and restrict decrypt rights accordingly.

- Rotate by setting the new value with `--secret` and running `up`. Changing the *provider* is a separate operation (`change-secrets-provider`).

Practice

1. Design config for a service with region, instance count, a list of allowed CIDRs, and a database password. Which keys go project-level, which stack-level, and which are secret?
2. A colleague creates a token with `random.RandomString` and marks `result` secret. What is still exposed, and what do you change?
3. Your team moves from Pulumi Cloud to an S3 DIY backend. What happens to secrets, and which provider would you pick?

Recall card

- `Pulumi.<stack>.yaml` holds per-stack config (commit it). `Pulumi.yaml config:` holds shared, plaintext, typed defaults.
- No-colon keys belong to the project. `Config("ns")` reads other namespaces.
- `require` fails loudly, `get` returns None, and `*_secret` keeps the marking.
- Secret-ness propagates through outputs, except inside `apply` callbacks and resource IDs.
- Use `--secrets-provider` at `stack init` and `stack change-secrets-provider` later. A passphrase is a key you must not lose.

Sources

- Pulumi, Configuration
- Pulumi, Secrets
- Pulumi, State and backends

Inputs, Outputs and the Dependency Graph

Inputs, Outputs and the Dependency Graph

A Pulumi program looks imperative: Python statements, run top to bottom. The infrastructure it describes is declarative: Pulumi decides what to create, in what order, and what to leave alone. The bridge between the two is a pair of types. **Inputs** are what you give a resource. **Outputs** are values the cloud reports back later. Every confusing moment for new Pulumi users ("why does my bucket name print as an object?") comes from treating an output as if it were already a value.

> **What was and was not run**
>
> `pulumi` and the provider SDKs were not available on the authoring box and installs were not allowed, so these Python snippets were not executed. They follow the current Pulumi documentation and SDK method names.

Learning goals

- Distinguish inputs, resource outputs and stack outputs
- Explain how passing an output into an input creates a dependency edge
- Transform outputs with `apply`, `Output.all`, `Output.format`, `Output.concat` and the JSON helpers, and reach nested values by lifting
- Recognise the "output used as a plain value" bug, and why resources must not be created inside `apply`

Inputs

An input is an argument to a resource, usually one parameter of the underlying cloud API call. Every input of type `Input[T]` also accepts a plain `T`:

```python
import pulumi_random as random

my_id = random.RandomId("mine", byte_length=8)   # 8 is a plain value used as an input
```

Python offers two spellings for object-shaped inputs: an `...Args` class or a dictionary typed as `...ArgsDict`. Both carry the same fields, and the dictionary form is shorter:

```python
import pulumi_aws as aws

repo = aws.ecr.Repository(
    "repo",
```

```
    image_tag_mutability="MUTABLE",
    image_scanning_configuration={"scan_on_push": True},
)
```

Outputs

An output is a property the provider reports **after** the resource exists, such as an ID, an ARN, an endpoint or a generated password. Outputs behave like futures. During a first `pulumi up` the value is unknown until the cloud answers. During `pulumi preview` it may never be known.

Pass an output straight into another resource's input and Pulumi records a dependency:

```
import pulumi_aws as aws
import pulumi_random as random

password = random.RandomPassword("password", length=16, special=True)

db = aws.rds.Instance(
    "db",
    instance_class="db.t3.micro",
    allocated_storage=64,
    engine="mysql",
    username="someone",
    password=password.result,      # output -> input: an edge in the graph
)
```

That edge tells Pulumi to create the password before the database, and on `destroy` to delete the database before the password. You wrote no ordering code. A program that wires outputs to inputs gets correct ordering for free, and `depends_on` is only for dependencies that no value carries.

Three similar names refer to different things:

Term	What it is
Input	An argument to a resource; accepts plain values or outputs
Resource output	A property of a resource, known once the provider reports it
Stack output	A value you `pulumi.export` at the end of a run for the CLI or a `StackReference`

Getting at the value

Because an output is not yet a value, you can't use ordinary language operations on it. `print(bucket.id)` or `"https://" + lb.dns_name` doesn't do what you expect. The SDK supplies the tools instead.

`apply`: transform one output

```
url = lb.dns_name.apply(lambda name: f"https://{name}")
```

The callback runs when the value is known and receives the plain value. Its return value becomes a new `Output`, which you can pass on as an input or export. Secret-ness and dependencies flow through.

`Output.all`: combine several

```
import pulumi

conn = pulumi.Output.all(server.name, database.name).apply(
    lambda args: f"Server=tcp:{args[0]}.database.windows.net;initial catalog={args[1]};"
)

# keyword form yields a dict
conn = pulumi.Output.all(server=server.name, db=database.name).apply(
    lambda a: f"Server=tcp:{a['server']}.database.windows.net;initial catalog={a['db']};"
)
```

Helpers: skip the lambda

Most `apply` calls build a string or some JSON. The helpers do that directly and are easier to read:

```
import json
import pulumi

url = pulumi.Output.format("https://{0}/api", lb.dns_name)
arn_path = pulumi.Output.concat(bucket.arn, "/*")

policy = pulumi.Output.json_dumps({
    "Version": "2012-10-17",
    "Statement": [{
        "Effect": "Allow",
        "Principal": "*",
        "Action": ["s3:GetObject"],
        "Resource": [arn_path],          # outputs allowed inside the structure
    }],
})

parsed = pulumi.Output.json_loads(some_json_output)
```

`json_dumps` accepts a structure that has outputs nested inside it. A `json.dumps` inside an `apply` would make you resolve each one by hand first.

Lifting: reach into nested outputs

Properties and indexes can be read straight off an output, with no `apply`. Pulumi "lifts" the access:

```
record_name = cert.domain_validation_options[0].resource_record_name
```

Without lifting this would be `cert.domain_validation_options.apply(lambda o: o[0].resource_record_nam`

Two rules that prevent most bugs

1. Never treat an output as a plain value. Printing it, concatenating it with `+`, using it in an `if`, or formatting it with an f-string outside `apply` gives you the wrapper object, not the value. Python's `Output` warns when converted to a string, and the result ends up in resource names and tags looking like garbage. Use `apply`, `format`, `concat` or `json_dumps`.

2. Don't create resources inside `apply` or `all`. Pulumi's documentation says to avoid it wherever possible. Resources created in a callback don't appear in `pulumi preview` unless the value is already known, so the preview stops matching what `up` will do. Reviewers then approve a plan that leaves things out. If the *number* of resources depends on a value, read that value from configuration (known at preview), not from another resource's output.

Resource identity

Wiring resources means passing *some* identifier, and the inputs differ in which one they want. A resource has four identities: its logical name (the first constructor argument), its physical name at the provider, its physical ID (`resource.id`), and its Pulumi URN. Many inputs want an ARN, an endpoint or a name instead of the ID. A type mismatch at preview usually means you passed the wrong identity. Check the input's documentation.

Diagnosis

Symptom	Cause	Fix
A name or tag contains text about calling `__str__` on an `Output`	Output used in an f-string or `str()`	`Output.format`/`concat`, or build it in `apply`
Preview shows fewer resources than `up` creates	Resources created inside `apply`	Drive counts from config. Create resources at top level
Resources created in the wrong order	The dependency is not carried by any value	Pass the output as an input, or `depends_on` as a last resort

Symptom	Cause	Fix
Type error passing `bucket` where a string is expected	Passed the resource, not a property	`bucket.id`, `bucket.arn` or `bucket.bucket`, per the input's docs
JSON policy has a placeholder where an ARN belongs	`json.dumps` on a structure containing outputs	`Output.json_dumps`

Practice

1. Build an output URL `https://<dns>/health` from a load balancer's `dns_name` three ways: `apply`, `Output.format` and `Output.concat`. Which reads best?
2. A colleague loops `for i in range(int(count_output))`. Explain why this fails and rewrite it to use config.
3. Write an S3 bucket policy that references two buckets' ARNs, using only `Output.json_dumps`.

Recall card

- Inputs accept plain values or outputs. Outputs are future values reported by the provider.
- Output to input means a dependency edge. Ordering for create and destroy is automatic.
- `apply` handles one output, `Output.all` several, and `format`/`concat`/`json_dumps`/`json_loads` are the common shortcuts. Lifting reads nested fields.
- Never treat an output as plain, and never create resources inside `apply`.

Sources

- Pulumi, Inputs and outputs
- Pulumi, Accessing single output values with apply
- Pulumi, Accessing multiple output values with all

Unit Testing Pulumi Programs with Mocks

Unit Testing Pulumi Programs with Mocks

After reading this chapter, you will be able to write fast, isolated unit tests for Pulumi infrastructure programs using Pulumi's built-in Python mock engine and standard test frameworks such as `unittest` and `pytest`. You will learn how to intercept cloud resource allocations and API invokes, validate configuration invariants and tagging policies, catch security misconfigurations before running a cloud preview, and handle Python asynchronous test execution correctly.

> **Verified environment**
>
> The code listings in this chapter were verified with Python 3.14.8, Pulumi Python SDK `3.273.0`, `pulumi-aws 7.52.1`, and `pytest 9.2.0`. No Pulumi CLI installation or cloud credentials are required to execute these unit tests.

Why test with mocks?

Deploying infrastructure to a cloud sandbox to verify logic takes minutes per test run, requires active AWS credentials, and incurs cloud billing. Even a local dry run like `pulumi preview` requires an authenticated session, network access to cloud APIs, and a state backend.

Unit testing with mocks solves a different problem: verifying that your program generates the intended resource definitions, flags, and relationships before contacting a provider. Pulumi executes your program in an in-memory test harness where:

- Cloud API calls are intercepted without making network connections.
- Mock handlers return synthesized IDs and attributes.
- Tests execute in milliseconds within standard CI pipelines.
- Security and compliance regressions are caught directly on developer machines.

Testing with mocks does not validate that AWS will accept an argument or that an IAM policy grants sufficient runtime privileges; that remains the domain of integration tests. Mocks validate the logic of *your* program.

The Pulumi mock runtime

The Pulumi Python SDK provides the `pulumi.runtime.Mocks` base class. When a Pulumi program instantiates resources or invokes provider functions, the SDK passes those requests to a registered subclass of `Mocks`.

A complete mock implementation overrides two methods:

```
class pulumi.runtime.Mocks:
    def new_resource(self, args: pulumi.runtime.MockResourceArgs) -> list:
        ...

    def call(self, args: pulumi.runtime.MockCallArgs) -> dict:
        ...
```

Method	Intercepts	Argument fields	Expected return value
`new_resource`	Resource declarations (`aws.s3.Bucket`, `aws.ec2.Instance`, etc.)	`args.typ`: resource type token (e.g. `aws:ec2/instance:Instance`); `args.name`: logical name in Pulumi program`args.inputs`: dictionary of input properties`args.provider`: provider reference`args.id_`: existing resource ID if any	A two-element list: `[resource_id, state_dict]`
`call`	Function invokes (`aws.ec2.get_ami`, `aws.get_caller_identity`, etc.)	`args.token`: function token (e.g. `aws:ec2/getAmi:getAmi`); `args.args`: dictionary of call arguments`args.provider`: provider reference	A dictionary containing output properties of the invoke

Once you define a mock class, you register it with `pulumi.runtime.set_mocks(MyMocks(), preview=False)`.

Registering mocks before program execution

The call to `pulumi.runtime.set_mocks()` must happen **before** your infrastructure module executes. If a resource or function runs before mocks are registered, the SDK attempts to connect to a live Pulumi engine RPC server and immediately fails.

Because Pulumi resources instantiate upon module import, you should not import your infrastructure module (`__main__` or your stack file) at the top of your test file. Instead, configure mocks inside the test setup phase and evaluate the program dynamically using `runpy.run_path()`.

Async testing with `IsolatedAsyncioTestCase`

Every Pulumi resource property is an `Output[T]`, which behaves like an asynchronous promise. To assert against values inside an `Output`, your test must inspect the output through `.apply()`.

The recommended pattern in modern Python combines `unittest.IsolatedAsyncioTestCase` with the `@pulumi.runtime.test` decorator:

1. Subclass `unittest.IsolatedAsyncioTestCase`.
2. In `setUp()`, invoke `pulumi.runtime.set_mocks(MyMocks(), preview=False)`.
3. Load the infrastructure program using `runpy.run_path("__main__.py")`.
4. Decorate each test method with `@pulumi.runtime.test`.
5. Return the resolved `Output` from the test method so the runner awaits its assertions.

Python 3.14 event loop requirements

On Python 3.14 and later, calling `asyncio.get_event_loop()` without an active event loop in the main thread raises `RuntimeError: There is no current event loop in thread 'MainThread'`. If `pulumi.runtime.set_mocks()` is called at module level outside of an active event loop, the test runner fails immediately.

Using `unittest.IsolatedAsyncioTestCase` manages the event loop lifecycle automatically for each test method. If you use a plain function-based test harness, you must call `asyncio.set_event_loop(asyncio.new_event_loop())` before calling `set_mocks()`.

Lab: testing an AWS infrastructure stack

We will define an AWS lab project containing an EC2 instance, a security group, and an S3 bucket. The initial version contains an intentional security flaw: an SSH ingress rule open to the entire internet (`0.0.0.0/0`). We will write unit tests to catch this policy violation and verify resource metadata.

Project dependencies

```
pulumi>=3.273.0,<4
pulumi-aws>=7.52.0,<8
pytest
```

The infrastructure program (v1 with security flaw)

The program queries an Amazon Linux 2023 AMI, configures a security group for web traffic and administrative SSH, provisions an EC2 instance, and creates an S3 bucket for access logs.

```
import pulumi
import pulumi_aws as aws

ami = aws.ec2.get_ami(
    most_recent=True,
    owners=["amazon"],
```

```
    filters=[
        aws.ec2.GetAmiFilterArgs(
            name="name",
            values=["al2023-ami-*-x86_64"],
        )
    ],
)

web_sg = aws.ec2.SecurityGroup(
    "web-sg",
    description="Allow HTTP and SSH access",
    ingress=[
        aws.ec2.SecurityGroupIngressArgs(
            protocol="tcp",
            from_port=80,
            to_port=80,
            cidr_blocks=["0.0.0.0/0"],
        ),
        aws.ec2.SecurityGroupIngressArgs(
            protocol="tcp",
            from_port=22,
            to_port=22,
            cidr_blocks=["0.0.0.0/0"],  # Security violation: SSH open to world
        ),
    ],
)

instance = aws.ec2.Instance(
    "web",
    instance_type="t3.micro",
    ami=ami.id,
    vpc_security_group_ids=[web_sg.id],
    tags={
        "Name": "web",
        "team": "platform",
    },
)

bucket = aws.s3.Bucket(
    "logs",
    tags={
        "team": "platform",
    },
)

pulumi.export("instance_id", instance.id)
pulumi.export("bucket_arn", bucket.arn)
```

The test suite

The test suite defines a mock implementation that satisfies both `new_resource` and `call`. It then runs assertions against tags, AMI resolution, and security group ingress CIDR blocks.

```
import runpy
import unittest
import pulumi


class InfraMocks(pulumi.runtime.Mocks):
    def new_resource(self, args: pulumi.runtime.MockResourceArgs):
        state = {
            **args.inputs,
            "arn": f"arn:aws:resource:::{args.name}",
        }
        return [f"{args.name}_id", state]

    def call(self, args: pulumi.runtime.MockCallArgs):
        if args.token == "aws:ec2/getAmi:getAmi":
            return {
                "id": "ami-0123456789abcdef0",
                "architecture": "x86_64",
            }
        return {}


class TestInfrastructure(unittest.IsolatedAsyncioTestCase):
    def setUp(self):
        pulumi.runtime.set_mocks(InfraMocks(), preview=False)
        self.program = runpy.run_path("__main__.py")

    @pulumi.runtime.test
    def test_instance_tags(self):
        instance = self.program["instance"]

        def check(tags):
            self.assertIsNotNone(tags, "Instance must have tags defined")
            self.assertEqual(tags.get("Name"), "web")
            self.assertEqual(tags.get("team"), "platform")

        return instance.tags.apply(check)

    @pulumi.runtime.test
    def test_instance_ami(self):
        instance = self.program["instance"]
```

```
        def check(ami_id):
            self.assertEqual(ami_id, "ami-0123456789abcdef0")

        return instance.ami.apply(check)

    @pulumi.runtime.test
    def test_security_group_ssh_restricted(self):
        web_sg = self.program["web_sg"]

        def check(ingress_rules):
            for rule in ingress_rules or []:
                from_port = rule.get("from_port") or rule.get("fromPort")
                to_port = rule.get("to_port") or rule.get("toPort")
                cidrs = rule.get("cidr_blocks") or rule.get("cidrBlocks") or []
                if from_port <= 22 <= to_port:
                    self.assertNotIn(
                        "0.0.0.0/0",
                        cidrs,
                        "Security group must not allow SSH from 0.0.0.0/0",
                    )

        return web_sg.ingress.apply(check)

    @pulumi.runtime.test
    def test_bucket_tags(self):
        bucket = self.program["bucket"]

        def check(tags):
            self.assertIsNotNone(tags, "Bucket must have tags defined")
            self.assertEqual(tags.get("team"), "platform")

        return bucket.tags.apply(check)


if __name__ == "__main__":
    unittest.main()
```

Running the tests against v1

From the project directory, with the virtual environment active (`source .venv/bin/activate`), run pytest in quiet mode without warnings and with one-line tracebacks:

```
python -m pytest -q -p no:warnings --tb=line test_infra.py
```

The command produced the following output:

```
...F                                                                    [100%]
=================================== FAILURES ===================================
E   AssertionError: '0.0.0.0/0' unexpectedly found in ['0.0.0.0/0'] : Security group must no
/tmp/lab/.venv/lib/python3.14/site-packages/pulumi/runtime/stack.py:211: AssertionError: '0.
=========================== short test summary info ============================
FAILED test_infra.py::TestInfrastructure::test_security_group_ssh_restricted
1 failed, 3 passed in 0.91s
```

Notice that three tests pass, but `test_security_group_ssh_restricted` raises an `AssertionError` identifying that port 22 is exposed to `0.0.0.0/0`. The reported location is inside the Pulumi SDK (`pulumi/runtime/stack.py`), not your test file. The assertion runs inside `.apply()`, and the mock runtime raises it again when it waits for outstanding work. Without `--tb=line`, you get a long traceback through the SDK's event loop, so rely on the assertion *message* to say what broke.

Naming the test file explicitly also matters. Once `fixed/test_infra.py` exists, a bare `pytest` run from the project root collects both copies and stops with an "import file mismatch" error, because the two modules share a basename.

Fixing the infrastructure flaw

To resolve the violation, we restrict port 22 access to the internal corporate CIDR `10.0.0.0/8`. Here is the complete corrected program.

```python
import pulumi
import pulumi_aws as aws

ami = aws.ec2.get_ami(
    most_recent=True,
    owners=["amazon"],
    filters=[
        aws.ec2.GetAmiFilterArgs(
            name="name",
            values=["al2023-ami-*-x86_64"],
        )
    ],
)

web_sg = aws.ec2.SecurityGroup(
    "web-sg",
    description="Allow HTTP and SSH access",
    ingress=[
        aws.ec2.SecurityGroupIngressArgs(
            protocol="tcp",
            from_port=80,
            to_port=80,
            cidr_blocks=["0.0.0.0/0"],
```

```
        ),
        aws.ec2.SecurityGroupIngressArgs(
            protocol="tcp",
            from_port=22,
            to_port=22,
            cidr_blocks=["10.0.0.0/8"],  # Fixed: restricted to internal CIDR
        ),
    ],
)

instance = aws.ec2.Instance(
    "web",
    instance_type="t3.micro",
    ami=ami.id,
    vpc_security_group_ids=[web_sg.id],
    tags={
        "Name": "web",
        "team": "platform",
    },
)

bucket = aws.s3.Bucket(
    "logs",
    tags={
        "team": "platform",
    },
)

pulumi.export("instance_id", instance.id)
pulumi.export("bucket_arn", bucket.arn)
```

Run the tests inside the `fixed` directory by reusing `test_infra.py`:

```
cd fixed && cp ../test_infra.py . && python -m pytest -q -p no:warnings --tb=line test_infra
```

The command produced the following output:

```
....                                                                     [100%]
4 passed in 0.95s
```

All four unit tests now execute and pass without contacting AWS or requiring the Pulumi CLI.

How mock outputs resolve

When writing assertions inside `check` callbacks, remember how Pulumi outputs behave in tests:

1. **Synthetic State**: When `new_resource` returns `[id, state]`, the SDK populates the created resource's properties with the keys provided in `state`. If your mock does not include an expected output field (for example, `public_ip`), accessing that attribute yields `None` or an unknown output.
2. **Preview Mode**: The parameter `preview=False` in `pulumi.runtime.set_mocks(MyMocks(), preview=False)` treats all mocked resource inputs as known. If set to `True`, computed values behave as unknowns, mirroring `pulumi preview`. For unit testing configuration policies, `preview=False` is standard.
3. **Property Normalization**: Input property names in Pulumi resources often use Python snake_case (such as `cidr_blocks`), but provider states may store camelCase keys (`cidrBlocks`). Checking both forms in your assertions ensures resilient tests across SDK schema versions.

Pitfalls

Mistake	Manifestation	Remedy
Importing `__main__` at module top-level	Tests crash with RPC connection errors to the Pulumi engine daemon	Register mocks in `setUp()` and load the program dynamically using `runpy.run_path()`
Calling `set_mocks` at module level on Python 3.14+	`RuntimeError: There is no current event loop in thread 'MainThread'`	Use `unittest.IsolatedAsyncioTestCase` so each test runs inside an established event loop
Forgetting to return the output promise from `@pulumi.runtime.test`	Tests exit prematurely without evaluating assertions in `.apply()`	Always return `resource.property.apply(check)` from the decorated test method
Asserting on resource outputs not defined in `new_resource`	Attribute resolves to `None` or an empty value	Include expected output attributes (e.g. `arn`, `id`) in the dictionary returned by `new_resource`
Attempting to validate provider behavior with unit mocks	Test passes, but cloud deployment fails due to invalid parameters	Use unit mocks to verify program declarations; use integration tests for provider compliance

Summary

- Pulumi unit tests intercept resource definitions and function calls entirely in memory, requiring neither the Pulumi CLI nor cloud provider credentials.
- The `pulumi.runtime.Mocks` class provides `new_resource` to simulate resource states and `call` to simulate provider lookup functions.
- Mocks must be configured using `pulumi.runtime.set_mocks()` before executing infrastructure definitions.

- `unittest.IsolatedAsyncioTestCase` provides clean event loop isolation on modern Python versions (including Python 3.14).
- Tests inspect resource state by passing assertions into `Output.apply()` inside methods decorated with `@pulumi.runtime.test`.

Sources

- Pulumi, Unit Testing with Mocks
- Pulumi, Testing Strategies
- Pulumi Registry, aws.ec2.getAmi API reference
- Python Software Foundation, unittest.IsolatedAsyncioTestCase documentation

Component Resources

Component Resources

After this page you can group related resources into a typed Python component resource, parent its children correctly, know which resource options reach those children, expose a secret output, validate arguments before anything is registered, unit-test the component with Pulumi's mocks, and rename it later without replacing what it manages.

Thesis

As infrastructure programs expand, instantiating low-level cloud primitives repeatedly leads to duplication, inconsistent security defaults, and brittle code. A plain Python function that returns resources lets you reuse code, but Pulumi itself never sees the grouping: the resources land flat in the stack, with nothing in the state or the CLI that ties them together.

Component resources solve this by inheriting from `pulumi.ComponentResource`. They introduce a custom type token into the resource graph, group child resources under a single parent identity, and give you one object to pass options to and to unit-test.

Why components? (vs functions returning resources)

When structuring reusable infrastructure constructs, developers often start by wrapping resource definitions inside standard Python functions. While simple, helper functions have structural limitations compared to `ComponentResource` classes.

```
# Helper function approach
def create_service_credentials(name: str, service: str):
    user = random.RandomPet(f"{name}-user", length=2)
    pwd = random.RandomPassword(f"{name}-password", length=24)
    return user, pwd
```

Comparing the two approaches across key architectural dimensions highlights why custom component resources are preferred for reusable infrastructure patterns:

Aspect	Helper function	Component resource
State	Each resource sits directly under the stack	One parent entry with the children nested under it
CLI tree view	A flat list	A subtree under the component

Aspect	Helper function	Component resource
Options	Pass `opts` to every resource yourself	Several options set on the component (such as `providers` and `protect`) are inherited by its children
Type identity	None; you get back a tuple	Its own type token, such as `acme:index:ServiceCredentials`
Child-wide changes	Edit every call site	`transforms` on the component can rewrite each child as it registers

Helper functions leave child resources floating at the root level of the Pulumi stack graph. A component gives them an owner, so previews and diffs read in the same units you think in.

The type token

Every Pulumi resource has a type token of the form `package:module:type`. Pulumi's own component guide uses `index` as the module for a component defined at the top level of a package, and this page follows that convention:

```
<package>:index:<ComponentName>
```

So in `acme:index:ServiceCredentials`, `acme` is your package or organisation namespace, `index` is the module, and `ServiceCredentials` is the type name.

The type token is part of the component's URN. Every child's URN contains it too, as you will see in the test output below. Change the token after deploying and Pulumi sees a new component with new children: it plans to create those and delete the old ones, unless you add an alias (covered later on this page).

The component

The following module defines `ServiceCredentials`, a component resource grouping a random pet name and password generation for a target service.

```
"""A component resource that groups the random resources one service needs."""
import pulumi
import pulumi_random as random


class ServiceCredentialsArgs:
    def __init__(self, service: str, password_length: int = 24):
        if password_length < 16:
            raise ValueError("password_length must be at least 16")
```

```
        self.service = service
        self.password_length = password_length


class ServiceCredentials(pulumi.ComponentResource):
    username: pulumi.Output[str]
    password: pulumi.Output[str]

    def __init__(self, name: str, args: ServiceCredentialsArgs,
                 opts: pulumi.ResourceOptions | None = None):
        super().__init__("acme:index:ServiceCredentials", name, None, opts)

        child = pulumi.ResourceOptions(parent=self)

        suffix = random.RandomPet(f"{name}-user", length=2, separator="-",
                                  opts=child)
        password = random.RandomPassword(f"{name}-password",
                                         length=args.password_length,
                                         special=False, opts=child)

        self.pet = suffix
        self.username = suffix.id.apply(lambda pet: f"{args.service}-{pet}")
        self.password = pulumi.Output.secret(password.result)

        self.register_outputs({
            "username": self.username,
            "password": self.password,
        })
```

Code Walk-Through

1. **Argument Class (`ServiceCredentialsArgs`)**: Holds the component's inputs and validates them before any resource exists. A `password_length` below 16 raises `ValueError` right away.
2. **Component Inheritance (`pulumi.ComponentResource`)**: The component extends `ComponentResource` and sets its outputs as class-level type annotations (`pulumi.Output[str]`).
3. **`super().__init__` Call**: Initializes the underlying Pulumi resource engine with the type token `"acme:index:ServiceCredentials"`, the logical resource name, no input properties (`None`), and optional `pulumi.ResourceOptions`.
4. **Parenting Child Resources**: `child = pulumi.ResourceOptions(parent=self)` creates a shared options instance. Passing `opts=child` to `random.RandomPet` and `random.RandomPassword` nests them under this component instance.
5. **Dynamic Naming**: Child resource logical names are dynamically formatted using `f"{name}-user"` and `f"{name}-password"`. With hard-coded child names, a second instance in the same stack would try to register duplicate URNs.
6. **Secret Output Wrappers**: `pulumi.Output.secret(password.result)` marks the generated output as sensitive, so Pulumi encrypts it in the state and masks it in CLI output.

7. `self.register_outputs(...)`: Tells Pulumi the component has finished registering its children and records its outputs.

`register_outputs` and what breaks without it

Pulumi's component guide puts it this way: calling `register_outputs` at the end of the constructor "signals Pulumi that the process of creating the component resource has completed". The dictionary you pass is stored as the component's outputs in the stack state. The child resources have their own state and are not affected.

Call it at the end of every component constructor, even with an empty dictionary. Leave it out and Pulumi has no explicit signal that the component is finished, and the component's outputs are never recorded.

Options that do and do not apply

A component has no cloud provider behind it, so options that change provider behaviour have no direct effect on the component itself. Pulumi's resource options reference lists which options a component passes on to its children:

Option set on the component	Inherited by its children?
`providers`, `provider`	Yes (but `provider` has no effect on the component itself; Pulumi recommends `providers` for components)
`protect`, `retain_on_delete`	Yes
`aliases`, `deleted_with`	Yes
`transforms`	Yes
`depends_on`	No
`ignore_changes`, `custom_timeouts`, `replace_on_changes`, `hooks`	No

For an option that is not inherited, set it on the children yourself, or use a transform on the component to add it to each child as that child registers:

```
# Add ignore_changes to every RandomPassword created under the component
def ignore_password_length(args: pulumi.ResourceTransformArgs):
    if args.type_ == "random:index/randomPassword:RandomPassword":
        return pulumi.ResourceTransformResult(
            props=args.props,
            opts=pulumi.ResourceOptions.merge(
                args.opts, pulumi.ResourceOptions(ignore_changes=["length"])),
        )
    return None
```

```
creds = ServiceCredentials(
    "api", ServiceCredentialsArgs(service="billing"),
    opts=pulumi.ResourceOptions(transforms=[ignore_password_length]))
```

This snippet follows the transforms documentation and was not run here.

Visualizing the resource tree

> **Note**
>
> **Not run here.** The Pulumi CLI is not installed on the machine used for this page, so no `pulumi preview` or `pulumi up` was run. The tree below shows the shape of that output (an illustration, not a captured run). The URN further down is real; it comes from the unit tests later on this page.

`pulumi up` draws the component as a parent with its children indented beneath it:

```
Type                                 Name            Status
pulumi:pulumi:Stack                  my-project-dev
  acme:index:ServiceCredentials      api             created
     random:index:RandomPet          api-user        created
     random:index:RandomPassword     api-password    created
```

A child's URN records the hierarchy by putting its parent's type token in front of its own type, joined with `$`:

```
urn:pulumi:stack::project::acme:index:ServiceCredentials$random:index/randomPet:RandomPet::a
```

Testing component resources

Unit testing component resources requires mocking the Pulumi runtime engine calls.

Complete Test Suite

The following listing tests the `ServiceCredentials` component without calling any real cloud endpoints.

```
import asyncio

import pulumi

# Python 3.14 no longer creates an event loop on demand; Pulumi's mocks need one.
asyncio.set_event_loop(asyncio.new_event_loop())
```

```
class Mocks(pulumi.runtime.Mocks):
    calls = []

    def new_resource(self, args: pulumi.runtime.MockResourceArgs):
        Mocks.calls.append((args.typ, args.name, args.custom))
        outputs = dict(args.inputs)
        if args.typ == "random:index/randomPet:RandomPet":
            return [f"{args.name}_id", outputs]
        if args.typ == "random:index/randomPassword:RandomPassword":
            outputs["result"] = "x" * int(args.inputs["length"])
        return [f"{args.name}_id", outputs]

    def call(self, args: pulumi.runtime.MockCallArgs):
        return {}


pulumi.runtime.set_mocks(Mocks(), preview=False)

import pytest  # noqa: E402
from credentials import ServiceCredentials, ServiceCredentialsArgs  # noqa: E402


@pulumi.runtime.test
def test_children_are_parented_to_the_component():
    creds = ServiceCredentials("api", ServiceCredentialsArgs(service="billing"))

    def check(args):
        urn, pet_urn, username = args
        print("component URN:", urn)
        print("child URN:    ", pet_urn)
        print("username:     ", username)
        for typ, name, custom in Mocks.calls:
            print(f"registered {typ} name={name} custom={custom}")
        assert urn.endswith("acme:index:ServiceCredentials::api")
        assert pet_urn.endswith("::acme:index:ServiceCredentials$random:index/randomPet:Rand
        assert username == "billing-api-user_id"

    return pulumi.Output.all(creds.urn, creds.pet.urn, creds.username).apply(check)


@pulumi.runtime.test
def test_password_is_secret_and_long_enough():
    creds = ServiceCredentials("db", ServiceCredentialsArgs(service="orders",
                                                            password_length=32))
```

```
    def check(args):
        password, is_secret = args
        assert len(password) == 32
        assert is_secret

    return pulumi.Output.all(creds.password,
                             pulumi.Output.from_input(
                                 creds.password).is_secret()).apply(check)


def test_short_passwords_are_rejected():
    with pytest.raises(ValueError, match="at least 16"):
        ServiceCredentialsArgs(service="x", password_length=8)
```

Two Critical Testing Pitfalls

1. Python 3.14 Event Loop Initialization

Since Python 3.14, `asyncio.get_event_loop()` raises `RuntimeError` when there is no current event loop, instead of creating one. Pulumi's mock setup asks for the current loop, so on Python 3.14 the tests fail before they start unless one has been set. Here is `python -m pytest -q -s test_credentials.py` with the `set_event_loop` line removed:

```
    raise RuntimeError('There is no current event loop in thread %r.'
E   RuntimeError: There is no current event loop in thread 'MainThread'.
ERROR test_credentials.py - RuntimeError: There is no current event loop in t...
1 error in 0.18s
```

To resolve this, explicitly instantiate and set an event loop before invoking `set_mocks`:

```
asyncio.set_event_loop(asyncio.new_event_loop())
```

2. Race Conditions in Mock Assertions

In the first version of the test, the `apply` hung off `creds.urn` alone. It ran as soon as the component's URN resolved, before the children had registered, so `Mocks.calls` held only the component. Wait on child outputs as well before you inspect what was registered:

```
return pulumi.Output.all(creds.urn, creds.pet.urn, creds.username).apply(check)
```

Running the Tests

The versions used (`python -c "import sys, pulumi, pytest; print(sys.version.split()[0], pulumi.__version__, pytest.__version__)"`):

```
Python 3.14.8
pulumi                                    3.273.0
pulumi-random                             4.21.6
pytest                                    9.2.0
```

Run the suite with `python -m pytest -q -s test_credentials.py`:

```
component URN: urn:pulumi:stack::project::pulumi:pulumi:Stack$acme:index:ServiceCredentials:
child URN:     urn:pulumi:stack::project::acme:index:ServiceCredentials$random:index/randomP
username:      billing-api-user_id
registered acme:index:ServiceCredentials name=api custom=False
registered random:index/randomPet:RandomPet name=api-user custom=True
...
3 passed in 0.11s
exit status: 0
```

In the output, `Mocks.new_resource` sees both kinds of resource: - The **component resource** (`custom=False`, type `acme:index:ServiceCredentials`). - The **child resources** (`custom=True`, types `random:index/randomPet:RandomPet` and `random:index/randomPassword:RandomPassword`).

Evolving a component resource

Components get renamed and reorganised. Each of those changes alters URNs, and a changed URN looks to Pulumi like a delete and a create unless you tell it otherwise.

Using Aliases for Component Renames

If a component's name or type token changes, give the old identity as an alias:

```python
# Alias old component type token to prevent recreation during updates
alias_option = pulumi.Alias(type_="acme:index:OldServiceCredentials")
super().__init__(
    "acme:index:ServiceCredentials",
    name,
    None,
    opts=pulumi.ResourceOptions.merge(opts, pulumi.ResourceOptions(aliases=[alias_option]))
)
```

State Migrations for Child Restructuring

When a child moves to a different parent, alias its old parent. `pulumi.Alias(parent=...)` accepts either a resource or a URN string:

```python
child_alias = pulumi.Alias(parent=old_parent_urn)  # URN string of the old parent
suffix = random.RandomPet(
    f"{name}-user",
    length=2,
    opts=pulumi.ResourceOptions(parent=self, aliases=[child_alias])
)
```

An alias tells Pulumi that the new URN is the same resource it already tracks, so it updates the state instead of replacing the resource. Aliases set on a component are inherited by its children, so a component rename carries the children with it. Run `pulumi preview` and check for zero replacements before you `up`. (Aliases were not exercised here; there is no CLI on this machine.)

Common mistakes

Anti-Pattern / Error	Consequence	Correct Pattern
Omitting `parent=self` on child resources	Children sit at the stack root and inherit nothing from the component	Pass `opts=pulumi.ResourceOptions(parent=sel` to every child
Hard-coding child logical names	Duplicate URN errors when instantiating multiple instances of the component	Prefix child names with the component instance name (`f"{name}-child"`)
Forgetting `self.register_outputs({...})`	Pulumi gets no explicit signal that the component is done, and its outputs are not recorded	Call `self.register_outputs(...)` at the end of the constructor
Passing `provider=` to a component	No effect on the component itself	Use `providers=[my_provider]` (or a map) on components
Changing type token without an `Alias`	Pulumi recreates component and all nested child resources	Pass old type tokens in `aliases=[Alias(type_="...")]`
Asserting `Mocks.calls` without waiting on children	Test runs before child resources register in mock list	Include child outputs in `Output.all(...)` before asserting calls

Practice

1. Modify `credentials.py` to add an optional `prefix` string argument to `ServiceCredentialsArgs`. If supplied, prepend the prefix to `username`. Add a unit test in `test_credentials.py` asserting that prefixes format correctly.
2. Add a `connection_string` output built with `pulumi.Output.format`, check that it stays secret, and include it in `register_outputs`.
3. Write a test showing that `password_length=16` (the boundary) is accepted and gives a 16-character password under the mocks.

Key takeaways

- Components subclass `pulumi.ComponentResource` and group related child infrastructure resources into a typed abstraction.
- Use type tokens of the form `<package>:index:<Name>`. Changing one changes every URN under it, so add an alias when you do.
- Child resources must specify `parent=self` and incorporate `name` into their logical names to preserve hierarchy and prevent URN collisions.
- End every constructor with `self.register_outputs(...)`.
- `providers`, `protect`, `aliases` and `transforms` reach the children; `depends_on` and `ignore_changes` do not, so use a transform for those.
- Under mocks, `new_resource` sees the component (`custom=False`) as well as its children (`custom=True`). Python 3.14+ test suites must set an `asyncio` event loop explicitly before calling `set_mocks`.

Sources

- Pulumi, Build a Component
- Pulumi, Component Resources Concept Guide
- Pulumi, Resource options (and component resources)
- Pulumi, Transforms
- Pulumi, Resource Aliases Guide
- Pulumi, Unit Testing Pulumi Programs
- Python Documentation, asyncio Event Loop Changes in Python 3.14

Workflows

Automation API: drive pulumi up from Python

Automation API: drive pulumi up from Python

After this page you can embed Pulumi deployments directly inside Python applications, manage stacks and workspaces programmatically using `LocalWorkspace` and `Stack`, run previews, updates, and destroys from code without calling the CLI binary directly, pass dynamic inline program functions, set stack configuration, and parse deployment outputs cleanly.

Thesis

The standard workflow for Pulumi relies on shell invocation: an engineer or a CI/CD pipeline executes `pulumi preview` or `pulumi up` via CLI commands. While effective for traditional deployments, CLI-driven operations break down when infrastructure management must be embedded inside larger application workflows, SaaS platforms, developer portals, or dynamic orchestration systems.

The Pulumi Automation API solves this problem by exposing the Pulumi engine as a standard programming library. Rather than executing shell scripts or wrapping CLI commands in subprocess calls, Python programs interact with infrastructure via strongly typed methods like `stack.up()` and `stack.destroy()`. This converts Pulumi from a CLI tool into an embedded deployment engine.

CLI vs. Automation API

Executing Pulumi through standard CLI workflows differs fundamentally from using the Automation API across several architectural boundaries:

Dimension	Pulumi CLI	Automation API
Control plane	Interactive terminal or shell scripts	Native application code (Python, Go, Node.js, .NET)
Program source	Static disk files in a directory	Disk files, or dynamic inline Python functions
Configuration	Static `Pulumi.<stack>.yaml` files	Programmatic configuration via typed methods
Output processing	Raw CLI text stdout or JSON string parsing	Strongly typed output objects (`OutputValue`)
Concurrency	Single process CLI executions	Multi-tenant background processing, queuing, and web APIs

Through the Automation API, infrastructure operations behave like standard SDK method calls, enabling automated testing, custom self-service portals, and runtime resource provisioning.

Key Abstractions: Workspace, Program, and Stack

The Automation API centers on three core concepts:

1. **Workspace**: Represents the environment where Pulumi operates. It handles plugin installation, backend state configuration, credentials, secrets providers, and environment variables. `LocalWorkspace` is the primary workspace implementation for local machine execution.
2. **Program**: The Pulumi infrastructure code to be executed. This can be specified as a local file directory path (an existing Pulumi project) or directly as an inline Python function (`InlineProgram`).
3. **Stack**: The isolated unit of deployment. A `Stack` instance binds a `Workspace` together with a `Program` and a specific stack name (e.g., `dev` or `prod`). All operations (`up`, `preview`, `destroy`, `refresh`) are executed against a `Stack` object.

Defining an Inline Program

With standard Pulumi, code resides in a `__main__.py` file. The Automation API allows you to define infrastructure using standard Python functions passed directly to `create_or_select_stack`.

The following script defines a small infrastructure program in Python and sets up a local workspace stack.

```
"""Drive Pulumi deployment using the Automation API with an inline program."""
import sys
from pulumi_service import create_stack


def pulumi_program():
    """Inline Pulumi program defining infrastructure resources."""
    import pulumi
    import pulumi_random as random

    # Register a random pet name resource
    pet = random.RandomPet("db-server-name", length=2, separator="-")

    # Export outputs to the stack state
    pulumi.export("server_name", pet.id)


def main():
    stack = create_stack("dev", pulumi_program)
    print("Selecting and updating stack...")
```

```
    # Run preview
    preview_res = stack.preview(on_output=sys.stdout.write)
    print(f"Preview change summary: {preview_res.change_summary}")

    # Run update
    up_res = stack.up(on_output=sys.stdout.write)
    print(f"Update summary: {up_res.summary.result}")

    # Retrieve output value
    server_name = up_res.outputs["server_name"].value
    print(f"Deployed server name: {server_name}")


if __name__ == "__main__":
    main()
```

The underlying deployment engine configuration and stack creation logic are encapsulated in a helper module:

```
"""Helper functions for managing Pulumi stacks via Automation API."""
from typing import Callable
from pulumi.auto import LocalWorkspace, Stack, ConfigValue


def create_stack(stack_name: str, program: Callable[[], None]) -> Stack:
    """Create or select an existing Pulumi stack with inline program logic."""
    project_name = "automation-demo"

    # Initialize or load stack with inline program definition
    stack = LocalWorkspace.create_or_select_stack(
        stack_name=stack_name,
        project_name=project_name,
        program=program,
    )

    # Set up plugin dependencies
    stack.workspace.install_plugin("random", "v4.16.0")

    # Configure stack values programmatically
    stack.set_config("aws:region", ConfigValue(value="us-west-2"))

    return stack
```

Code Walk-Through

1. **`LocalWorkspace.create_or_select_stack`**: Instantiates a `Stack` object. If the stack `dev` under `automation-demo` does not exist, it creates it in the workspace backend; otherwise, it loads the existing state.
2. **`program=pulumi_program`**: Accepts a zero-argument Python function containing resource allocations and `pulumi.export` statements.
3. **`stack.workspace.install_plugin(...)`**: Automates provider plugin installation, ensuring the execution host has the correct binary providers downloaded prior to running updates.
4. **`stack.set_config(...)`**: Sets project or provider configuration programmatically using `ConfigValue` objects without needing to edit YAML files manually.
5. **`stack.preview()` and `stack.up()`**: Executes the deployment phases asynchronously or synchronously. The `on_output` parameter streams stdout directly to stdout streams or custom logger functions.
6. **`up_res.outputs`**: Contains a dictionary of `OutputValue` objects exported by `pulumi.export` during program execution.

Executing Updates and Reading Outputs

> **Note**
>
> **Not run here.** The Pulumi Python SDK and CLI are not installed on the build runner for this environment. The command and output block below illustrate expected execution output during a real run.

To run the script:

```
python auto-up.py
```

The execution output produced by the script above follows this structure:

```
<<OUTPUT:auto-up-run>>
```

Programmatic Destroy and Stack Teardown

Managing the lifecycle of short-lived environments (such as preview environments for pull requests or test fixtures) requires reliable teardown mechanisms. The Automation API exposes `stack.destroy()` and `stack.workspace.remove_stack()`.

The following script demonstrates automated destruction and removal of stack resources:

```
"""Tear down resources programmatically using Automation API."""
import sys
from pulumi.auto import LocalWorkspace, Stack
```

```
def empty_program():
    """Empty program placeholder for selecting existing stacks."""
    pass


def destroy_environment(stack_name: str, project_name: str):
    """Destroy all stack resources and remove state registration."""
    stack = LocalWorkspace.select_stack(
        stack_name=stack_name,
        project_name=project_name,
        program=empty_program,
    )

    print(f"Destroying resources in stack '{stack_name}'...")
    destroy_result = stack.destroy(on_output=sys.stdout.write)
    print(f"Destroy summary: {destroy_result.summary.result}")

    print(f"Removing stack registration for '{stack_name}'...")
    stack.workspace.remove_stack(stack_name)
    print("Stack removed successfully.")


if __name__ == "__main__":
    destroy_environment("dev", "automation-demo")
```

Key Considerations for Teardown

- **`select_stack` vs `create_or_select_stack`**: Use `select_stack` when interacting with pre-existing stacks where program code is either loaded from disk or unnecessary during teardown.
- **`stack.destroy()`**: Runs Pulumi's deletion workflow, invoking resource provider deletion hooks in reverse dependency order.
- **`remove_stack()`**: Clears stack state entries from the backend after resources are destroyed. Attempting to call `remove_stack()` on a stack that still manages active resources raises an error.

Working with Existing Disk-Based Projects

In addition to inline Python functions, `LocalWorkspace` can load standard Pulumi project directories containing existing `Pulumi.yaml` and source code files.

```
"""Drive a file-based Pulumi project using LocalWorkspace."""
import sys
from pulumi.auto import LocalWorkspace, LocalWorkspaceOptions
```

```python
def run_disk_project(project_dir: str, stack_name: str):
    """Load and deploy a standard Pulumi project from disk."""
    # Configure workspace settings referencing target directory
    opts = LocalWorkspaceOptions(
        work_dir=project_dir,
    )

    stack = LocalWorkspace.create_or_select_stack(
        stack_name=stack_name,
        opts=opts,
    )

    print(f"Refreshing state for stack '{stack_name}'...")
    stack.refresh(on_output=sys.stdout.write)

    print("Running update...")
    result = stack.up(on_output=sys.stdout.write)

    for key, value in result.outputs.items():
        print(f"Output: {key} = {value.value} (secret={value.is_secret})")


if __name__ == "__main__":
    run_disk_project("./infra-project", "staging")
```

Setting `work_dir` points `LocalWorkspace` to a directory containing a valid `Pulumi.yaml` file, executing the infrastructure code defined in that project's entrypoint.

Common Pitfalls

Pitfall	Cause	Solution
Missing CLI Binary	The Automation API still invokes the underlying `pulumi` CLI binary under the hood.	Ensure `pulumi` is installed in system `PATH` or specify `PULUMI_HOME` in workspace settings.
Uninstalled Plugins	Provider plugins are not automatically inherited from other projects when running inline programs.	Call `stack.workspace.install_plugin(name, version)` explicitly before running `stack.up()`.
Stale State on Removal	Calling `stack.workspace.remove_stack()` before calling `stack.destroy()`.	Always invoke `stack.destroy()` first to remove actual cloud resources before unregistering the stack.

Pitfall	Cause	Solution
Uncaptured Secrets	Accessing `OutputValue.value` directly on secret outputs logs plaintext values in application outputs.	Inspect `OutputValue.is_secret` property before logging or printing stack outputs.

Key takeaways

- The **Automation API** exposes the Pulumi engine as a standard software library, allowing infrastructure deployments to be driven programmatically from code.
- **`LocalWorkspace`** manages Pulumi project settings, plugin downloads, configuration values, and environment variables without shell scripts.
- **`InlineProgram`** functions allow defining Pulumi resources directly inside application Python functions, removing the need for separate project files on disk.
- **`Stack`** objects expose core lifecycle commands—`preview()`, `up()`, `refresh()`, and `destroy()`—and return strongly typed output objects.

Sources

- Pulumi Automation API Concept Guide
- Getting Started with Automation API in Python
- Pulumi Package Registry

Pulumi ESC: environments, secrets, and dynamic credentials

Pulumi ESC: environments, secrets, and dynamic credentials

After this page you can explain what Pulumi ESC adds on top of stack configuration, sketch an environment YAML that holds secrets and non-secrets, open an environment for a `pulumi up`, and know where ESC stops and stack config begins.

> **What was and was not run**
>
> `pulumi` and the ESC CLI were not available on the authoring box and installs were not allowed. Layouts, commands and YAML below follow the current Pulumi ESC documentation (pulumi.com/docs/esc/). Marked **Not run here** where a live CLI would be required.

Thesis

Stack configuration (Part 01) is the right place for values that differ per stack: instance size, region, a feature flag. It is the wrong place to reinvent an organisation-wide secrets store, to duplicate the same database password into twenty stacks, or to mint short-lived cloud credentials by hand.

Pulumi ESC — Environments, Secrets, and Configuration — is Pulumi's hierarchical environment layer. An *environment* is a named document (YAML) that can import other environments, pull secrets from providers, and expose a flat set of values to `pulumi`, CI, or any process that opens it. Stacks then consume those values instead of re-storing them.

Mental model

```
              ESC environment
imports: [acme/prod-shared]
values:
  aws:     { login: { fn::open::aws-login: … } }   # dynamic creds
  dbPassword: { fn::secret: "…" }                  # encrypted secret
  region: us-east-1                                # plain config

                          pulumi config env add  /  esc run

               Pulumi stack (dev / prod)
               program reads config + OIDC/env
```

Layer	Owns	Lifetime
ESC environment	Shared secrets, provider logins, org defaults	Independent of any one stack
Stack configuration	Per-stack overrides (`instanceType`, feature flags)	Bound to that stack's state
Program code	How resources are declared	Versioned in git

An environment document

Environments live in the Pulumi Cloud (or a self-hosted backend that supports ESC) and are edited as YAML. A minimal shape looks like this (illustrative; **Not run here**):

```yaml
values:
  region: us-east-1
  dbPassword:
    fn::secret: "replace-me"
  # Dynamic credentials are opened when the environment is opened,
  # not stored as long-lived static keys in the document.
  aws:
    login:
      fn::open::aws-login:
        oidc:
          duration: 1h
          roleArn: arn:aws:iam::123456789012:role/pulumi-deploy
          sessionName: pulumi-esc
```

Points that matter in review:

1. **`fn::secret`** marks a value as secret so ESC encrypts it at rest and redacts it in logs, the same class of protection stack secrets get.
2. **`fn::open::…`** providers mint *dynamic* values (OIDC cloud logins, vault reads) at open time. The document stores *how* to open them, not the raw credential.
3. **Imports** let a leaf environment pull in `acme/prod-shared` and override only what must differ.

Opening an environment for a stack

Two common operator paths (commands from the docs; **Not run here**):

```
# Attach an environment to a stack so `pulumi up` sees its values
pulumi config env add acme/prod-shared

# Or run an arbitrary command with the environment opened into the process
esc run acme/prod-shared -- pulumi up
```

Once attached, values appear to the program through the usual config APIs (`pulumi.Config().require_secret("`
provider env vars from the login provider, and so on). Prefer reading secrets through the config/secret APIs rather than printing them.

ESC vs stack secrets (chapter 03)

Concern	Stack `pulumi config set --secret`	ESC environment
Scope	One stack	Many stacks / many projects via imports
Rotation	Edit that stack	Edit once in the environment
Dynamic cloud login	Manual / external OIDC wiring	First-class `fn::open::` providers
Local plaintext `Pulumi.<stack>.yaml`	Encrypted ciphertext in the stack file	Values live in the ESC service; stack keeps a reference

Keep per-stack knobs in stack config. Promote anything shared or short-lived into ESC.

Common mistakes

- Duplicating the same `fn::secret` into every stack YAML "for convenience" — you lose rotation and audit.
- Treating ESC as a replacement for least-privilege IAM: an opened OIDC role should still be scoped to the deploy task.
- Logging opened values in CI (`echo $DB_PASSWORD`) — redaction in ESC does not protect a shell that prints them.
- Assuming every backend supports ESC the same way — confirm your Pulumi Cloud / self-host version before designing around imports.

Exercises

1. Sketch an environment that imports `acme/base` and overrides only `region`.
2. List three values in your current project that should move from stack config to ESC, and three that should stay.
3. (**Not run here**) On a machine with ESC configured, run `esc open <env> --format json` and confirm secrets are redacted in the default view.

Summary

ESC is the shared, hierarchical layer for secrets and dynamic credentials; stack configuration remains the per-stack dial. Open an environment for a deploy, keep long-lived static keys out of git, and rotate once at the environment rather than twenty times at the stack.

Sources

- Pulumi ESC documentation
- Get started with ESC
- Environments